Proyecto Final
Análisis de Datos y Machine Learning

Predicción con modelo de Machine Learning
API REST · Docker · MongoDB

Grupos de 2 – 3 integrantes

Índice

1. Objetivo del proyecto
2. Requisitos generales
3. Dataset
4. Análisis Exploratorio de Datos (EDA)
5. Limpieza y preprocesamiento de datos
6. Normalización / Estandarización
7. Creación y entrenamiento del modelo
8. Evaluación del modelo
9. Exportación del modelo
10. API REST con FastAPI
11. Base de datos MongoDB
12. Dockerización
13. Testing
14. Estructura del repositorio
15. Entregables
16. Criterios de evaluación
17. Anexo – Recursos y enlaces útiles

1. Objetivo del proyecto

Desarrollar un proyecto completo de Data Science / Machine Learning que cubra todo el ciclo de vida de un modelo predictivo: desde la obtención y exploración de los datos, pasando por la limpieza, transformación, entrenamiento y evaluación del modelo, hasta su puesta en producción a través de una API REST construida con FastAPI, desplegada en contenedores Docker y respaldada por una base de datos MongoDB que registre cada petición de predicción.

El objetivo es experimentar un flujo real de trabajo de un científico de datos e ingeniero ML, integrando análisis estadístico, programación, despliegue y persistencia de datos en un único proyecto cohesionado.

2. Requisitos generales

- Los grupos serán de entre 2 a 3 integrantes.
- El dataset debe obtenerse de Kaggle u otra fuente pública reconocida (UCI ML Repository, data.gov, etc.).
- El proyecto debe contener al menos un notebook (.ipynb) con todo el análisis, limpieza, entrenamiento y evaluación.
- El modelo entrenado debe exportarse a un archivo (.pkl, .joblib o .h5) y cargarse en la API.
- La API REST debe estar construida con FastAPI (Python).
- Cada petición de predicción debe almacenarse en MongoDB (entrada + resultado + timestamp).
- Todo el sistema debe estar dockerizado y poder levantarse con un solo comando: docker-compose up.
- El repositorio debe tener al menos las ramas main y develop.
- El código debe seguir buenas prácticas: funciones documentadas, código limpio, nombres descriptivos.

3. Dataset

El equipo debe elegir un dataset de Kaggle (u otra fuente pública) que cumpla las siguientes condiciones:

- Tener al menos 1000 filas y 10 columnas.
- Contener una variable objetivo numérica (para regresión) o categórica (para clasificación).
- Incluir datos "sucios": valores nulos (NaN), posibles duplicados y/o columnas con tipos mixtos.
- Contener tanto variables numéricas como categóricas (texto).

Ejemplos de datasets válidos:

Dataset	Fuente	Target	Tipo
Used Car Price Prediction	Kaggle	Price	Regresión
Medical Insurance Cost	Kaggle	charges	Regresión
Titanic – Machine Learning from Disaster	Kaggle	Survived	Clasificación
Heart Disease UCI	Kaggle / UCI	target	Clasificación
Laptop Price Prediction	Kaggle	Price	Regresión

El equipo puede proponer cualquier otro dataset siempre que cumpla los requisitos anteriores. Debe comunicarse al profesor antes de empezar para su validación.

4. Análisis Exploratorio de Datos (EDA)

Antes de modificar nada, es fundamental entender los datos. El EDA debe estar documentado en el notebook con explicaciones en celdas Markdown y debe incluir, como mínimo:

4.1 Inspección inicial

- Mostrar las primeras filas del dataset (df.head()).
- Dimensiones del dataset: número de filas y columnas (df.shape).
- Tipos de datos de cada columna (df.info()): identificar columnas numéricas (int64, float64) y categóricas (object/string).
- Estadísticas descriptivas (df.describe()): media, mediana, desviación estándar, min, max, cuartiles.

4.2 Análisis de la variable objetivo

- Histograma de la variable objetivo para ver su distribución.
- Indicar si la distribución está sesgada (cola a la derecha o izquierda).
- Comparar media vs mediana para detectar sesgo.
- Si el target es categórico: gráfico de barras con la frecuencia de cada clase y comprobar si está balanceado.

4.3 Análisis de correlaciones

- Calcular la matriz de correlación de las variables numéricas con la variable objetivo.
- Mapa de calor (heatmap) que muestre las correlaciones. Se recomienda usar colores tipo "Purples": cuanto más intenso, mayor correlación.
- Scatter plots de las variables más correlacionadas vs. la variable objetivo.
- Identificar las variables más relevantes para el modelo.

💡 **Nota:** Todas las visualizaciones deben realizarse con matplotlib y/o seaborn. Cada gráfico debe tener título, etiquetas en los ejes y una breve explicación en una celda Markdown del notebook.

5. Limpieza y preprocesamiento de datos

Los datos "crudos" casi nunca están listos para entrenar un modelo. Es necesario limpiarlos y transformarlos. Este apartado debe documentar cada paso en el notebook con explicaciones claras.

5.1 Eliminación de duplicados

- Detectar filas duplicadas con df.duplicated().sum().
- Eliminar duplicados con df.drop_duplicates().
- Documentar cuántas filas se eliminaron.

5.2 Tratamiento de valores nulos (NaN)

Se deben aplicar estrategias diferenciadas:

1. **Eliminar.** Columnas con más del 40-50% de nulos:

   Si una columna tiene demasiados valores faltantes, no aporta información útil. Se elimina directamente.

2. **Rellenar con un valor por defecto.** Columnas categóricas (texto) con nulos:

   Se usa df[col].fillna("Desconocido") o fillna("None"). Así el modelo trata la ausencia de dato como una categoría más.

3. **Rellenar con la mediana.** Columnas numéricas con nulos:

   La mediana es más robusta que la media frente a outliers (valores extremos). Se usa df[col].fillna(df[col].median()).

- Visualizar los nulos con un gráfico de barras indicando el porcentaje por columna.
- Verificar al final que no quedan nulos: df.isnull().sum().sum() == 0.

5.3 Tratamiento de variables categóricas

Los modelos de ML no pueden trabajar con texto directamente. Hay que convertir las columnas categóricas a valores numéricos. Se permiten dos estrategias:

- **Para variables nominales (sin orden).** One-Hot Encoding (pd.get_dummies()):

  Ejemplo: "Neighborhood" → crea una columna binaria por cada barrio.

- **Para variables ordinales (con orden lógico).** Encoding ordinal (mapeo manual):

  Ejemplo: "ExterQual" con valores Po < Fa < TA < Gd < Ex → se mapea a 1, 2, 3, 4, 5.

5.4 Eliminación de outliers (opcional pero recomendado)

Los outliers son valores extremos que pueden distorsionar el modelo. Se recomienda identificarlos mediante scatter plots o box plots y eliminarlos si son claramente erróneos (por ejemplo, una casa de 5000 m² que cuesta $50,000).

6. Normalización / Estandarización

Las variables numéricas suelen tener escalas muy diferentes. Por ejemplo, una columna puede ir de 1 a 10 y otra de 0 a 500,000. Esto puede hacer que el modelo dé más peso a las variables con valores grandes.

6.1 StandardScaler (estandarización)

Transforma cada variable para que tenga media = 0 y desviación estándar = 1. Es la técnica más habitual para regresión lineal, SVM y redes neuronales.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_val_sc = scaler.transform(X_val)
```

6.2 Log-transform del target (si aplica)

Si la variable objetivo tiene una distribución sesgada (cola larga a la derecha), se recomienda aplicar una transformación logarítmica: y = np.log1p(target). Esto hace que la distribución sea más simétrica y muchos modelos funcionan mejor. Al hacer predicciones, se revierte la transformación con np.expm1(prediccion).

⚠ **Importante:** El scaler se ajusta SOLO con los datos de entrenamiento (fit_transform). Los datos de validación y test solo se transforman (transform), nunca se hace fit sobre ellos. Esto evita "data leakage" (fuga de información).

7. Creación y entrenamiento del modelo

Con los datos limpios y normalizados, se procede a entrenar el modelo predictivo. El equipo debe implementar al menos dos modelos y comparar sus resultados.

7.1 División de datos (train / validation / test)

Se deben separar los datos en al menos dos conjuntos:

- Entrenamiento (train): 80% de los datos. Se usa para entrenar el modelo.

- Validación (val): 20% de los datos. Se usa para evaluar el rendimiento antes del envío final.

```
from sklearn.model_selection import train_test_split
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

7.2 Modelos a implementar

Se debe entrenar al menos un modelo base y uno avanzado:

Modelo base (obligatorio):

- Regresión Lineal (para regresión) o Regresión Logística (para clasificación).
- Sirve como baseline: un punto de referencia para saber si los modelos avanzados mejoran.

Modelo avanzado (obligatorio, elegir al menos uno):

- Random Forest (RandomForestRegressor / RandomForestClassifier)
- Gradient Boosting (GradientBoostingRegressor / GradientBoostingClassifier)
- XGBoost (requiere instalar la librería xgboost)
- Ridge / Lasso (para regresión con regularización)

Para cada modelo se debe mostrar: código de entrenamiento, métricas de evaluación y una breve comparación indicando cuál es mejor y por qué.

7.3 Ingeniería de variables (opcional pero valorado)

Crear nuevas variables a partir de las existentes puede mejorar significativamente el modelo. Ejemplos:

- Superficie total = sótano + planta 1 + planta 2.
- Edad de la vivienda = año de venta - año de construcción.
- Variables booleanas: tiene piscina (sí/no), tiene garaje (sí/no).
- Interacciones: calidad × superficie.

8. Evaluación del modelo

El modelo debe evaluarse con métricas cuantitativas y visualizaciones. Las métricas dependen del tipo de problema:

Para problemas de REGRESIÓN:

Métrica	Qué mide	Interpretación
MAE (Mean Absolute Error)	Error medio en las mismas unidades que el target.	Cuanto menor, mejor. Fácil de interpretar.
RMSE (Root Mean Squared Error)	Error medio penalizando errores grandes.	Cuanto menor, mejor. Más sensible a outliers que MAE.

R^2 (Coeficiente de determinación)	Proporción de varianza explicada por el modelo.	1.0 = perfecto, 0.0 = no explica nada. >0.8 suele ser bueno.

Para problemas de CLASIFICACIÓN:

Métrica	Qué mide	Interpretación
Accuracy	Porcentaje de predicciones correctas.	Útil si las clases están balanceadas.
Precision / Recall / F1	Equilibrio entre falsos positivos y negativos.	Usar F1 si las clases están desbalanceadas.
Confusion Matrix	Tabla de predicciones vs. realidad.	Permite ver dónde falla el modelo visualmente.

Además de las métricas numéricas, se deben incluir gráficos:

- Gráfico de coeficientes o importancia de variables.
- Gráfico predicho vs. real (scatter plot con línea de referencia).
- Residuos del modelo (si es regresión).

9. Exportación del modelo

Una vez seleccionado el mejor modelo, debe exportarse a un archivo para poder cargarlo en la API sin necesidad de reentrenar.

Formatos permitidos:

- **pickle / joblib →** .pkl (pickle):

  El más habitual para modelos de scikit-learn.

- **joblib →** .joblib:

  Similar a pickle pero más eficiente para arrays grandes.

- **HDF5 / Keras →** .h5 / .keras:

  Solo si se usa TensorFlow/Keras.

Ejemplo con joblib:

```
import joblib

# Guardar
joblib.dump(modelo, "models/modelo_final.pkl")
joblib.dump(scaler, "models/scaler.pkl")
```

```
# Cargar
modelo = joblib.load("models/modelo_final.pkl")
scaler = joblib.load("models/scaler.pkl")
```

⚠ **Importante:** Si se usó un StandardScaler, también hay que exportarlo. La API necesita aplicar la misma transformación a los datos de entrada antes de predecir.

10. API REST con FastAPI

El modelo entrenado debe servirse a través de una API REST construida con FastAPI. La API recibe los datos de entrada (features de la vivienda, coche, paciente, etc.), ejecuta la predicción y devuelve el resultado.

10.1 Endpoints mínimos obligatorios

Método	Ruta	Descripción	Response
GET	/	Tel leva al swagger	200 + swagger
POST	/predict	Recibe features JSON, devuelve predicción.	200 + predicción
GET	/predictions	Lista todas las predicciones almacenadas en MongoDB.	200 + lista
GET	/predictions/{id}	Detalle de una predicción concreta.	200 + detalle

10.2 Ejemplo de petición POST /predict

```
// Request Body (JSON)
{
  "OverallQual": 7,
  "GrLivArea": 1500,
  "GarageCars": 2,
  "TotalBsmtSF": 800,
  "YearBuilt": 2005
}

// Response
{
  "id": "665a1b...",
  "prediction": 215000.50,
  "timestamp": "2025-06-01T12:30:00Z",
  "model_used": "GradientBoosting"
}
```

10.3 Requisitos técnicos

- Usar Pydantic models para validación de entrada (BaseModel).
- El modelo .pkl se carga UNA sola vez al arrancar la API (no en cada petición).
- Gestión de errores: devolver códigos HTTP apropiados (400 si faltan campos, 500 si falla el modelo).
- Documentación automática de Swagger disponible en /docs.

11. Base de datos MongoDB

Cada vez que se realice una predicción a través de la API, se debe almacenar un registro en MongoDB con la siguiente información:

- Datos de entrada (features enviadas por el usuario).
- Resultado de la predicción.
- Timestamp (fecha y hora de la petición).
- Nombre del modelo utilizado.
- (Opcional) Tiempo de inferencia en milisegundos.

11.1 Esquema de documento (ejemplo)

```
{
  "_id": ObjectId("665a1b..."),
  "input": {
    "OverallQual": 7,
    "GrLivArea": 1500,
    "GarageCars": 2
  },
  "prediction": 215000.50,
  "model": "GradientBoosting",
  "timestamp": ISODate("2025-06-01T12:30:00Z"),
  "inference_ms": 12.5
}
```

11.2 Requisitos técnicos

- Usar la librería pymongo o motor (versión asíncrona) para conectarse.
- La conexión a MongoDB se configura mediante variables de entorno (MONGO_URI).
- MongoDB se ejecuta en su propio contenedor Docker.
- Los datos deben persistir aunque el contenedor se elimine (usar volúmenes Docker).

12. Dockerización

Todo el sistema debe estar containerizado con Docker. El proyecto debe poder arrancarse con un solo comando:

```
docker-compose up --build
```

12.1 Contenedores requeridos

Servicio	Imagen base	Puerto
api	python:3.11-slim	8000
mongo	mongo:7	27017

12.2 Estructura del docker-compose.yml

```
version: "3.8"
services:
  api:
    build: ./api
    ports:
      - "8000:8000"
    environment:
      - MONGO_URI=mongodb://mongo:27017/predictions_db
    depends_on:
      - mongo

  mongo:
    image: mongo:7
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db

  mongo-express:  # Opcional: panel web para ver MongoDB
    image: mongo-express
    ports:
      - "8081:8081"
    environment:
      - ME_CONFIG_MONGODB_URL=mongodb://mongo:27017/
    depends_on:
      - mongo

volumes:
  mongo_data:
```

12.3 Dockerfile de la API (ejemplo)

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

13. Testing

El proyecto debe incluir pruebas automatizadas con PyTest. Las pruebas deben cubrir:

- Test del endpoint /predict: enviar datos válidos y verificar que devuelve una predicción numérica.
- Test de validación: enviar datos incompletos o inválidos y verificar que devuelve error 400/422.
- Test de /predictions: verificar que devuelve una lista.
- Test del modelo: cargar el modelo exportado y verificar que predice correctamente con datos conocidos.

Ejemplo de test con FastAPI TestClient:

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_predict():
    response = client.post("/predict", json={
        "OverallQual": 7,
        "GrLivArea": 1500,
        "GarageCars": 2
    })
    assert response.status_code == 200
    assert "prediction" in response.json()
```

Mínimo: 5 tests que pasen correctamente.

14. Estructura del repositorio

El repositorio debe seguir la siguiente estructura (adaptable según el proyecto):

```
proyecto/
├── README.md
├── docker-compose.yml
├── notebook/
│   ├── EDA_y_modelo.ipynb       # Notebook con todo el análisis
│   ├── data/
│   │   ├── train.csv
│   │   └── test.csv
│   └── models/
│       ├── modelo_final.pkl
│       └── scaler.pkl
├── api/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── main.py                  # Aplicación FastAPI
│   ├── models/
│   │   ├── modelo_final.pkl
```

```
│   │   └── scaler.pkl
│   ├── schemas/
│   │   └── prediction.py        # Pydantic models
│   ├── services/
│   │   └── predictor.py         # Lógica de predicción
│   ├── database/
│   │   └── mongo.py             # Conexión a MongoDB
│   └── tests/
│       └── test_api.py
└── docs/
    └── presentacion.pdf
```

📌 **Nota:** Al clonar el repositorio y ejecutar docker-compose up --build, todo el sistema debe levantarse correctamente sin pasos adicionales.

15. Entregables

4. Repositorio Git (GitHub/GitLab) con la estructura indicada y ramas main + develop.
5. Notebook (.ipynb) con EDA, limpieza, entrenamiento, evaluación y explicaciones en Markdown.
6. API REST funcional con FastAPI + modelo cargado.
7. docker-compose.yml que levante toda la infraestructura.
8. Tests automatizados con PyTest (mínimo 5 tests).
9. Diapositivas de la exposición oral (10-15 min por grupo).
10. README.md completo: descripción del proyecto, cómo ejecutarlo, tecnologías utilizadas, estructura del repo y capturas de pantalla.

16. Anexo – Recursos y enlaces útiles

Datasets

- Kaggle: https://www.kaggle.com/datasets
- UCI Machine Learning Repository: https://archive.ics.uci.edu/ml/index.php

Documentación

- FastAPI: https://fastapi.tiangolo.com/
- scikit-learn: https://scikit-learn.org/
- pandas: https://pandas.pydata.org/
- seaborn: https://seaborn.pydata.org/
- pymongo: https://pymongo.readthedocs.io/
- Docker Compose: https://docs.docker.com/compose/

Tutoriales recomendados

- Kaggle Learn – Intro to Machine Learning: https://www.kaggle.com/learn/intro-to-machine-learning
- FastAPI Tutorial oficial: https://fastapi.tiangolo.com/tutorial/
- Docker para principiantes: https://docs.docker.com/get-started/